Company Brain

Technical Architecture Document

Tech Stack · File Structure · Database Schema · Configuration

July 21, 2026

1. Recommended Tech Stack

Each choice below is deliberately the smallest/cheapest option that still meets a specific requirement surfaced during scoping — RBAC-aware retrieval, data ownership, citation traceability, and cost control.

Layer	Choice	Reasoning
LLM — primary	Claude Sonnet 5 + Claude Haiku 4.5	Sonnet handles agentic routing/synthesis where reasoning quality matters; Haiku handles high-volume cheap sub-tasks (chunk headers, entity extraction) to control cost
LLM — optional cost-sensitive path	Qwen3 4B / Gemma 3 4B (quantized, self-hosted)	Swappable backend for low-stakes, high-volume sub-tasks where a wrong output is easy to review; avoids per-token fees on the highest-volume calls
Agent framework	LangGraph	Explicit state machine for multi-step tool routing and retrieval loops — easier to debug and extend with new connectors than an implicit agent loop
Embeddings	Voyage AI (voyage-3)	Strong retrieval quality, native LangChain/LlamaIndex support; embedding cost is negligible next to generation cost
Vector database	Qdrant (self-hosted, Docker)	Fast similarity search with metadata filters, which is what makes RBAC-at-retrieval possible; self-hostable so the client keeps data ownership
Relational database	PostgreSQL	Handles structured metadata (users, projects, access, audit) and the lightweight knowledge-graph tables without needing a separate graph database at MVP scale
Cache / session store	Redis	Sub-millisecond reads for chat session memory, rate limiting, and background job queuing
Object storage	AWS S3	Durable, cheap storage for original files; keeps raw documents intact and independently exportable from the processed index
Backend API	FastAPI (Python)	Async-native, first-class fit with the Anthropic SDK and LangChain/LlamaIndex ecosystem, automatic OpenAPI docs
Document parsing	Docling (primary), PyMuPDF (fallback), Tesseract OCR	Layout-aware parsing preserves tables/structure; OCR fallback covers scanned documents
Frontend — admin & graph dashboard	Next.js/React + a force-directed graph library	One codebase serves both the RBAC admin console and the onboarding knowledge-graph visualization
Connectors	Slack Bolt SDK, Microsoft Graph API, Google Workspace API, Meta WhatsApp Business API + manual upload path	Official SDKs only — avoids the Terms-of-Service issues identified during scoping
Desktop agent	Python, packaged with PyInstaller	Lightweight, cross-platform, user-triggered sync

Layer	Choice	Reasoning
		rather than a silent background daemon
Deployment	Docker Compose on a single cloud VM (MVP)	Fast to stand up for a 15-day build; each service isolated and independently upgradable later
Evaluation	Ragas / custom Claude-as-judge scripts	Automated retrieval precision/recall and faithfulness scoring to catch regressions as the knowledge base grows

2. Project File & Folder Structure

A monorepo layout, split into four independently deployable pieces: the backend API/agent, the desktop agent, the frontend (admin console + graph dashboard), and the evaluation harness.

```
company-brain/
├── backend/
│   ├── app/
│   │   ├── main.py                      # FastAPI app entrypoint
│   │   ├── core/
│   │   │   ├── config.py                # env var loading, settings
│   │   │   └── security.py              # auth, JWT, RBAC helpers
│   │   ├── api/routes/
│   │   │   ├── chat.py                  # chat/query endpoint
│   │   │   ├── admin.py                 # project & access management
│   │   │   ├── ingestion.py             # manual upload, desktop agent sync endpoint
│   │   │   └── webhooks.py              # Slack/Teams/WhatsApp webhook receivers
│   │   ├── agents/
│   │   │   ├── router.py                # LangGraph tool-routing agent
│   │   │   ├── tools.py                 # search_slack, search_drive,
search_meetings, etc.
│   │   │   └── prompts.py
│   │   ├── connectors/
│   │   │   ├── slack_connector.py
│   │   │   ├── teams_connector.py
│   │   │   ├── gmail_connector.py
│   │   │   ├── drive_connector.py
│   │   │   └── whatsapp_upload.py
│   │   ├── ingestion/
│   │   │   ├── parsing.py               # Docling / PyMuPDF / OCR
│   │   │   ├── chunking.py              # structure-aware chunking
│   │   │   ├── contextual_retrieval.py  # LLM chunk-header generation
│   │   │   └── graph_extraction.py      # entity/relationship extraction for the
graph
│   │   ├── retrieval/
│   │   │   ├── vector_store.py          # Qdrant client wrapper
│   │   │   ├── hybrid_search.py         # BM25 + semantic
│   │   │   └── rbac_filter.py           # project/role filtering at query time
│   │   ├── models/                      # SQLAlchemy ORM models
│   │   ├── schemas/                     # Pydantic request/response schemas
│   │   └── db/
│   │       ├── session.py
│   │       └── migrations/              # Alembic migrations
│   ├── tests/
│   ├── requirements.txt
│   └── Dockerfile
├── desktop-agent/
│   ├── agent.py                         # folder scan, review flow trigger
```

```
│   ├── matcher.py                  # fuzzy matching against project profile
│   └── build/                      # PyInstaller build output
├── frontend/
│   ├── app/                        # Next.js app router
│   │   ├── chat/                   # optional web chat UI
│   │   ├── admin/                  # project & access management console
│   │   └── graph/                  # knowledge-graph visualization dashboard
│   ├── components/
│   └── package.json
├── eval/
│   ├── eval_set.json               # hand-labeled Q&A pairs with expected
sources
│   └── run_eval.py                 # Ragas / Claude-as-judge scoring
├── infra/
│   ├── docker-compose.yml
│   └── .env.example
├── scripts/
│   └── seed_projects.py
└── README.md
```

3. Database Schema

PostgreSQL holds all structured data — metadata, access control, session memory, audit trail, and the knowledge-graph tables. Vector embeddings themselves live in Qdrant; document_chunks.vector_id is the bridge between the two.

users

One row per employee who can use or administer Company Brain.

Field	Description
id	Primary key
email	Employee's work email; used to map Slack/Teams/WhatsApp identity to an internal user
name	Display name
role	employee / manager / hr / admin — coarse role, separate from project-level access
created_at	Account creation timestamp

Relationships: Referenced by project_assignments (which projects a user can access), conversations (their chat sessions), and audit_log (their query history).

projects

One row per project/client engagement that has its own scoped knowledge base.

Field	Description
id	Primary key
name	Project name, e.g. “Atlas Migration”
aliases	JSON array of alternate names/codenames used for file and message matching

Field	Description
client_name	External client this project relates to, if applicable
date_range_start / date_range_end	Rough active period, used by the desktop agent and connectors to scope what's relevant
created_by	Foreign key to users — who created the project record
created_at	Creation timestamp

Relationships: The central scoping entity: documents, graph_nodes, graph_edges, and project_assignments all reference a project. Nothing enters the knowledge base without belonging to exactly one project.

project_assignments

Join table that is the actual mechanism behind role-based access control — who can see which project's knowledge base.

Field	Description
id	Primary key
project_id	Foreign key to projects
user_id	Foreign key to users
assigned_by	Foreign key to users — must be an admin
assigned_at	Timestamp of the grant

Relationships: Many-to-many between users and projects. Every retrieval query checks this table before any chunk is returned — this is where access is enforced, not in the prompt.

documents

One row per ingested file, message thread, or export, before it's broken into chunks.

Field	Description
id	Primary key
project_id	Foreign key to projects
source	Enum: slack / drive / gmail / teams / whatsapp_upload / desktop_agent / manual_upload
source_ref	External identifier or permalink (e.g. Slack message link, Drive file ID)
title	Display name/subject of the document
file_path	S3 key where the original file is stored
uploaded_by	Foreign key to users
status	pending / parsed / embedded / failed — tracks pipeline progress
ingested_at	Timestamp

Relationships: Belongs to one project. Parent of document_chunks (one document produces many chunks). The source and source_ref fields are what power the clickable citation shown in every answer.

document_chunks

One row per structure-aware chunk produced during ingestion — the actual unit that gets embedded and retrieved.

Field	Description
id	Primary key
document_id	Foreign key to documents
chunk_index	Order of this chunk within the document
chunk_text	The raw chunk content
contextual_header	LLM-generated 1–2 sentence header prepended before embedding (contextual retrieval)
vector_id	Reference to the corresponding point in Qdrant
token_count	Used for cost tracking and context-window budgeting

Relationships: Belongs to one document. vector_id links this row to its embedding in Qdrant, so a retrieval hit can be traced back to Postgres metadata and from there to the original file.

graph_nodes

One row per entity (person, project, document, client) surfaced for the knowledge-graph visualization.

Field	Description
id	Primary key
project_id	Foreign key to projects — the graph is always scoped to a project
entity_type	person / project / document / client
entity_name	Canonical display name, after alias merging
aliases	JSON array of alternate names collapsed into this node
source_chunk_id	Foreign key to document_chunks — optional, the chunk this entity was extracted from

Relationships: Scoped to a project. Connected to other nodes via graph_edges. Traceable back to the originating chunk (and therefore document) for verification.

graph_edges

One row per relationship between two entities, e.g. “Priya works_on Atlas Migration.”

Field	Description
id	Primary key
project_id	Foreign key to projects

Field	Description
from_node_id / to_node_id	Foreign keys to graph_nodes
relationship_type	e.g. works_on, owns, mentioned_in, reports_to
source_chunk_id	Foreign key to document_chunks — where this relationship was extracted from

Relationships: Connects two graph_nodes within the same project. This is what the force-directed graph in the onboarding dashboard actually renders.

conversations

One row per chat session, so follow-up questions can retain context.

Field	Description
id	Primary key
user_id	Foreign key to users
channel	slack / whatsapp / teams
started_at / last_active_at	Session lifetime, used to expire stale sessions from Redis

Relationships: Belongs to one user. Parent of messages (one conversation has many turns).

messages

One row per turn in a conversation, both the employee's question and the agent's answer.

Field	Description
id	Primary key
conversation_id	Foreign key to conversations
role	user / assistant
content	Message text
cited_sources	JSON array of document/chunk references shown as citations, for assistant messages
created_at	Timestamp

Relationships: Belongs to one conversation. cited_sources references documents and document_chunks, giving a full audit trail from an answer back to its source.

audit_log

One row per query, independent of conversation threading — the compliance/accountability record.

Field	Description
id	Primary key

Field	Description
user_id	Foreign key to users
query_text	The question asked
sources_retrieved	JSON array of chunks/documents the retrieval layer returned before generation
timestamp	When the query was made

Relationships: References users. Deliberately denormalized/append-only so it remains a reliable record even if a conversation is later deleted.

eval_runs

One row per evaluation harness run, tracking answer quality over time.

Field	Description
id	Primary key
run_at	Timestamp of the run
model_used	Which LLM configuration was evaluated (e.g. Claude Sonnet, or the open-source fallback)
precision / recall	Retrieval quality scores against the hand-labeled eval set
faithfulness_score	How well answers stuck to retrieved content rather than the model's own knowledge
notes	Free text — what changed since the last run

Relationships: Standalone table, not referenced elsewhere; exists purely to track quality regressions release over release.

connector_credentials

One row per connected external account (Slack workspace, Google Workspace tenant, etc.).

Field	Description
id	Primary key
connector_type	slack / teams / gmail / drive / whatsapp
encrypted_token	OAuth token or API credential, encrypted at rest
connected_by	Foreign key to users — who authorized the connection
status	active / revoked / expired
connected_at	Timestamp

Relationships: Referenced by the ingestion layer whenever a connector job runs; not linked to a specific project since a single Slack/Drive connection typically spans multiple projects.

4. Environment Variables & Configuration Notes

4.1 Environment Variables

Variable	Purpose
ANTHROPIC_API_KEY	Claude API access for the main agent and Haiku sub-tasks
OPEN_SOURCE_LLM_ENDPOINT	URL of the self-hosted Qwen/Gemma inference server, if the cost-sensitive path is enabled
VOYAGE_API_KEY	Embedding provider for chunk vectors
DATABASE_URL	PostgreSQL connection string
REDIS_URL	Session memory, cache, and job-queue connection
QDRANT_URL / QDRANT_API_KEY	Vector store connection
AWS_ACCESS_KEY_ID / AWS_SECRET_ACCESS_KEY	Credentials for S3 raw-file storage
AWS_S3_BUCKET / AWS_REGION	Target bucket and region for file storage
SLACK_BOT_TOKEN / SLACK_SIGNING_SECRET	Slack connector authentication and webhook verification
MS_GRAPH_CLIENT_ID / MS_GRAPH_CLIENT_SECRET / MS_TENANT_ID	Microsoft Graph / Teams connector
GOOGLE_OAUTH_CLIENT_ID / GOOGLE_OAUTH_CLIENT_SECRET	Gmail / Drive connector
WHATSAPP_BUSINESS_TOKEN / WHATSAPP_PHONE_NUMBER_ID	Only required if the WhatsApp Business API is used for live business-conversation capture
JWT_SECRET	Signs admin console session tokens
ENVIRONMENT	dev / staging / prod flag, controls logging verbosity and safety checks
LOG_LEVEL	Logging verbosity

4.2 Configuration Notes

- Never commit .env — only .env.example belongs in the repo; use a secrets manager (AWS Secrets Manager, Doppler, etc.) in staging/production.
- Connector tokens should be scoped as narrowly as each platform allows, not one org-wide token with blanket access to every channel or drive.
- Postgres, Qdrant, and S3 should sit inside the client's own cloud account/VPC, consistent with the data-hosting decision made during scoping.

- Rotate the Anthropic API key and connector tokens on a schedule; audit_log doubles as a place to spot anomalous usage patterns.
- Enable S3 bucket versioning and a lifecycle policy so raw files are never silently lost or overwritten.
- If the open-source model path is enabled, it needs its own properly sized server — it should not be bolted onto the API server's resources.
- RBAC must be enforced inside rbac_filter.py at the retrieval layer, not left to the LLM prompt — this was a specific commitment made to the client.